

ssacurity-stm32-drive 트러블슈팅 README

이 문서는 OrinCar의 STM32F429 주행 제어기를 개발하면서 실제로 발생한 문제를 증상, 원인, 해결, 검증 및 재발 방지 기준으로 정리한 기록이다.

문서의 기준 상태는 다음과 같다.

- MCU/보드: STM32F429ZI / STM32F429I-DISC1
- 모터 드라이버: BTS7960
- 구동 모터: 12 V DC 모터와 2상 엔코더
- 조향: MG996R, 실제 조향센서 미장착
- 거리 센서: HC-SR04
- PC 벤치 통신: USART1 / ST-LINK VCP / COM11
- 최종 Jetson 통신: UART5 / PC12 TX / PD2 RX
- 인터페이스 문서 버전: V3
- UART wire protocol version: 0x02

상태 표시는 다음 의미다.

상태	의미
해결 완료	코드와 설정에 반영했고 빌드 또는 실차에서 확인함
임시 해결	현재 시험에는 사용할 수 있지만 바닥 주행이나 추가 센서 검증이 필요함
미검증	코드 또는 설계는 있으나 실제 하드웨어 결과가 아직 없음

1. 전체 사례 요약

ID	문제	핵심 원인	현재 상태
T01	역방향 모터가 돌지 않음	LPWM 핀과 보드 내장 기능 충돌	해결 완료
T02	UART는 ACCEPTED 인데 하드웨어가 움직이지 않음	수락 응답은 물리 동작 보증이 아님	해결 완료
T03	낮은 PWM에서 모터가 돌지 않음	모터/기어박스 정지 마찰과 데드존	해결 완료
T04	엔코더 Count가 0 또는 ±1만 반복	엔코더 VCC/GND 불량, 공통 기준전압 부재	해결 완료
T05	엔코더 한 바퀴 Count 추정이 크게 다름	한 바퀴 육안 측정과 기존 임시값 오류	해결 완료
T06	COM11에서 서보/모터 응답이 없음	활성 UART가 UART5인데 COM11은 USART1임	해결 완료
T07	servo-calibrate 가 없는 명령이라고 나옴	PC 도구와 저장소/펌웨어 버전 불일치	해결 완료
T08	텔레메트리가 21 byte여야 한다는 오류	구형 V1 파서로 26-byte V2 텔레메트리 해석	해결 완료

ID	문제	핵심 원인	현재 상태
T09	한 번 보낸 주행 명령이 곧 정지함	300 ms heartbeat watchdog과 재무장 규칙	해결 완료
T10	초음파 표적이 멀면 계속 센서 오류로 정지	NO_ECHO와 센서 오류를 같은 상태로 처리	해결 완료/제약 있음
T11	서보 중앙이 1500 us 또는 1300 us가 아님	실차 링크 중앙이 데이터시트 일반값과 다름	해결 완료
T12	서보 진단 종료 후 중앙으로 자동복귀	진단 종료 직후 일반 0도 명령이 다시 적용됨	해결 완료
T13	조향센서 없이 각도 명령을 만들 수 없음	실제 바퀴각 피드백 부재와 비선형 링크	임시 해결
T14	매뉴얼 치수로 아커만 계산이 맞지 않음	전체 길이/폭을 휠베이스/조향축 간격으로 오해	해결 완료
T15	형상값을 넣었는데 오도메트리가 갱신되지 않음	조향센서 유효 조건과 V2 오도메트리 송신 미연결	미검증/후속 작업

2. 빠른 증상별 점검 순서

UART 응답이 전혀 없을 때

1. COM 포트가 실제 ST-LINK VCP인지 확인
2. VEHICLE_COMM_USE_STLINK_VCP 값 확인
3. PC 도구 self-test 실행
4. 최신 펌웨어를 다시 빌드하고 플래시
5. 보드 리셋
6. echo 시험
7. 그래도 실패하면 TX/RX/DMA와 활성 UART 확인

```
py tools\uart_protocol_test.py self-test
py tools\uart_protocol_test.py echo --port COM11 --text PC
```

명령은 ACCEPTED인데 모터가 돌지 않을 때

1. 12 V 모터 전원 확인
2. BTS7960 R_EN/L_EN 확인
3. RPWM=PA0, LPWM=PA3 배선 확인
4. 20% 미만의 데드존 시험을 피함
5. +25%, -25%를 각각 시험
6. 엔코더 Count와 적용 Duty를 함께 확인

```
py tools\uart_protocol_test.py motor-test --port COM11 --percent 25 --duration 1000
py tools\uart_protocol_test.py motor-test --port COM11 --percent -25 --duration 1000
```

모터는 도는데 엔코더가 0일 때

1. 엔코더 모듈의 명세 전압에 맞는 VCC 확인
2. 엔코더 GND와 STM32 GND 공통 연결
3. A상=PB6, B상=PB7 확인
4. 모터를 끄고 손으로 바퀴를 회전
5. 손 회전 Count가 확인된 후 모터 시험

서보가 움직였다가 중앙으로 돌아올 때

1. 새 펌웨어가 플래시됐는지 확인
2. raw 진단 종료 시 PWM OFF 처리 여부 확인
3. 일반 CMD_STOP의 정상 중앙복귀와 진단 PWM OFF를 구분
4. PWM OFF 후 기구 힘으로 움직이는 현상은 토크 소멸 여부로 판단

3. 모터 트러블슈팅

T01. 역방향 모터가 돌지 않았던 문제

증상:

- 양수 모터 명령은 정상 동작
- 음수 명령도 UART에서 **ACCEPTED**
- 실제 모터는 역방향으로 회전하지 않음

초기 원인:

- 역방향 LPWM을 PA1/TIM5_CH2에 배정했다.
- PA1은 STM32F429I-DISC1의 온보드 자이로센서 INT1과 기판상 공유된다.
- **.ioc**에서 타이머 채널이 설정됐다는 사실만으로 보드에서 자유롭게 사용할 수 있다고 판단하면 안 된다.

해결 과정:

- 중간 단계에서 PA6/TIM13_CH1도 검토 및 사용했다.
- 최종 차량 배선과 프로젝트 설정은 PA3/TIM5_CH4로 통일했다.
- RPWM과 LPWM을 모두 TIM5의 20 kHz 채널로 구성했다.
- **.ioc**, **main.c**, MSP 초기화, **motor.c**, 실제 배선을 함께 변경했다.

현재 확정 매핑:

기능	STM32	타이머	BTS7960
정방향 PWM	PA0	TIM5_CH1 / 20 kHz	RPWM
역방향 PWM	PA3	TIM5_CH4 / 20 kHz	LPWM
Enable	PE2, PE3	GPIO	R_EN, L_EN

추가 주의:

- PA0은 보드 사용자 버튼과 공유되므로 주행 중 버튼을 누르지 않는다.
- PA3은 LCD의 B5 기능과 공유되지만 현재 프로젝트는 LCD를 초기화하지 않는다.
- LCD를 나중에 활성화하면 PA3 충돌을 다시 검토해야 한다.

근거:

- Git e8acfdc: fix: BTS7960 LPWM 핀을 PA3 TIM5_CH4로 변경
- ssacurity-stm32-drive.ioc
- Drivers/BSP/Src/motor.c

T02. ACCEPTED인데 모터가 움직이지 않는 문제

Motor test command: ACCEPTED는 다음 사실만 뜻한다.

프레임 형식 정상
CRC 정상
명령 범위 정상
현재 상태에서 명령 수락 가능

다음 사실은 보증하지 않는다.

12 V 전원이 실제로 공급됨
BTS7960 Enable이 High임
PWM 선이 올바르게 연결됨
모터가 정지 마찰을 이김
엔코더가 실제 회전을 감지함

따라서 수락 응답만으로 시험 성공을 판정하지 않고 다음 세 값을 함께 본다.

- 적용된 motor_duty_permille
- encoder_count 변화
- 시험 종료 후 Duty가 0인지

이 원칙 때문에 현재 encoder-test와 drive-test는 UART 응답뿐 아니라 텔레메트리의 Duty와 엔코더 방향까지 검증한다.

T03. 10~15% PWM에서 모터가 돌지 않는 문제

확인된 현상:

- 약 10~15%: 모터가 정지 마찰을 이기지 못하고 정지
- 약 20% 이상: 회전 시작

해결:

- 비영점 속도 명령에는 22% feed-forward를 먼저 적용한다.
- PI 출력은 22% 주변의 보정값으로 사용한다.
- 현재 구동 출력은 목표 방향 안에서 최대 40%로 제한한다.
- 진단 명령은 0% 또는 절댓값 20~60%만 허용한다.

현재 제어 상수:

```
Kp = 120
Ki = 20
Kd = 0
minimum running PWM = 22%
maximum running PWM = 40%
control period = 10 ms
```

과거 P 제어 시험 결과는 피드백 방향과 정·역 대칭성을 확인하는 근거로만 사용한다. 현재 PI 이득과 실제 차량 하중에서의 최종 성능은 다시 시험해야 한다.

4. 엔코더 트러블슈팅

T04. Count가 0 또는 ± 1 만 반복된 문제

증상:

- 모터는 회전하지만 `encoder-test`의 Count delta가 0
- 손으로 바퀴를 돌리면 `-1`, `0`, `+1` 부근에서만 불규칙하게 변화
- 한때 큰 누적 Count가 보였지만 이후 고정됨

실차에서 확인된 원인:

- 엔코더 VCC/GND가 정상적으로 연결되지 않았다.
- 특히 STM32와 엔코더의 GND 기준이 일치하지 않았다.

해결:

- 엔코더 모듈의 명세에 맞게 VCC를 재배선했다.
- 엔코더 GND와 STM32 GND를 공통으로 연결했다.
- A상과 B상은 각각 PB6/TIM4_CH1, PB7/TIM4_CH2에 유지했다.

교훈:

- A/B 선이 맞아도 공통 GND가 없으면 정상적인 디지털 레벨을 판정할 수 없다.
- 작은 ± 1 변화는 정상 펄스가 아니라 부유 입력이나 노이즈일 수 있다.
- 배선을 바꾸기 전에 모터를 끄고 손 회전 시험으로 입력단부터 확인한다.

T05. Counts per revolution 보정

한 바퀴 육안 측정은 시작점과 끝점 오차가 커서 처음 값이 부정확했다. 방향별 3회전을 측정해 평균했다.

```
정방향 3회전: +2473 count
역방향 3회전: -2464 count

정방향: 2473 / 3 = 824.33 count/rev
역방향: 2464 / 3 = 821.33 count/rev
평균:      822.83 count/rev
최종 설정: 823 count/rev
```

현재 타이어 명목 지름은 64 mm를 사용한다.

```
명목 둘레 = pi * 0.064 m
           = 0.20106 m

거리/count = 0.20106 / 823
            = 약 0.0002443 m
            = 약 0.2443 mm
```

현재 10 ms 속도 계산에서 1 Count는 약 24.43 mm/s이므로 저속 텔레메트리가 계단식으로 보이는 것은 정상적인 양자화 현상이다.

정방향 이동은 양수, 역방향 이동은 음수로 정의한다. 실제 부호가 반대라면 A/B 배선을 즉시 바꾸기보다 `Encoder_SetDirectionSign(-1)`을 이용해 차량 좌표계와 맞출 수 있다.

남은 검증:

- 64 mm는 명목 지름이다.
- 하중 상태에서 바닥을 실제 1 m 주행한 뒤 유효 구름 둘레를 보정해야 한다.

5. UART와 펌웨어 버전 트러블슈팅

T06. COM11이 맞는데 응답이 없던 문제

STM32에는 두 UART가 동시에 초기화되지만 `CommService`가 사용하는 활성 전송 경로는 하나다.

설정	활성 경로	용도
<code>VEHICLE_COMM_USE_STLINK_VCP=1U</code>	USART1 / PA9, PA10 / ST-LINK VCP	PC COM11 시험
<code>VEHICLE_COMM_USE_STLINK_VCP=0U</code>	UART5 / PC12 TX, PD2 RX	최종 Jetson

COM11은 ST-LINK VCP이므로 펌웨어가 UART5를 선택한 상태에서는 COM11이 정상이어도 응답이 없다.

해결:

- PC 벤치에서는 설정을 1U로 빌드하고 다시 플래시한다.
- 최종 Jetson에서는 0U로 빌드하고 PC12/PD2/GND를 사용한다.
- 설정을 바꾼 뒤에는 빌드만 하지 말고 반드시 보드에 플래시하고 리셋한다.

COM11 시험은 프로토콜과 제어 로직을 검증하지만 UART5 핀과 최종 배선을 검증하지는 않는다. 최종 이관 전 PC의 3.3 V USB-UART로 PC12/PD2 Echo를 한 번 확인해야 한다.

T07. `servo-calibrate`가 없는 명령이라고 나온 문제

증상:

```
invalid choice: 'servo-calibrate'
```

원인:

- 실행 중인 `tools/uart_protocol_test.py`가 새 명령이 추가되기 전 버전이었다.
- 또는 저장소는 갱신했지만 다른 디렉터리의 도구를 실행했다.

반대로 다음 메시지는 도구에는 명령이 있지만 펌웨어가 맞지 않을 때 주로 발생한다.

```
No valid V2 servo response received
Flash the updated firmware first
```

하지만 같은 메시지는 UART 포트가 잘못 선택됐을 때도 발생할 수 있으므로 다음 순서로 구분한다.

1. 현재 작업 디렉터리 확인
2. `--help`에서 명령 목록 확인
3. `self-test` 확인
4. 활성 UART 설정 확인
5. 최신 펌웨어 플래시
6. 보드 리셋 후 재시험

PC 도구와 STM32 펌웨어는 하나의 프로토콜 버전 세트로 관리해야 한다.

6. 프로토콜 트러블슈팅

T08. `TELEMETRY_DRIVE` payload is 26 bytes, expected 21

원인:

- STM32는 현재 26-byte V2 `TELEMETRY_DRIVE`를 송신했다.
- PC 도구가 구형 21-byte 구조체로 해석했다.
- UART 배선이나 엔코더 고장이 아니라 송수신 구조체 버전 불일치였다.

현재 26-byte 필드:

```
mcu_time_ms
target_speed_mm_s
measured_speed_mm_s
motor_duty_permille
```

```
steering_cmd_cdeg
steering_feedback_cdeg
encoder_count
yaw_cdeg
state
last_drive_seq
active_fault_bits
```

해결:

- Python과 STM32의 wire version을 0x02로 통일했다.
- Python은 `struct.calcsize()`로 26-byte 길이를 계산한다.
- 응답 프레임의 자체 SEQ와 요청 payload 안의 `request_sequence`를 구분했다.
- Golden Frame과 CRC self-test를 추가했다.

추가 충돌:

- 구형 V1의 0x82는 `TELEMETRY_ODOMETRY`였다.
- 현재 V2의 0x82는 `COMMAND_RESULT`다.
- 현재 활성 프로토콜에서 0x82를 오도메트리로 해석하면 안 된다.

T09. 한 번 보낸 명령이 300 ms 뒤 멈추는 문제

이 동작은 오류가 아니라 통신 단절 안전 설계다.

```
Jetson/PC: CMD_DRIVE를 20 Hz, 50 ms 주기로 계속 송신
STM32: 최신 유효 CMD_DRIVE 수신 시 watchdog 갱신
300 ms 동안 갱신 없음: PWM 0, SAFE_STOP, COMM_TIMEOUT
```

Timeout 이후에는 곧바로 비영점 명령을 보내도 거부된다. 먼저 다음 neutral 재무장 명령이 필요하다.

```
target_speed = 0
target_steering = 0
drive_enable = 0
```

그 다음부터 비영점 `CMD_DRIVE`를 다시 보낼 수 있다.

`CMD_STOP`도 재무장을 요구한다. 따라서 Jetson 제어기는 단발성 명령 전송기가 아니라 주기적인 command heartbeat 송신기로 구현해야 한다.

7. 초음파 안전 트리블슈팅

T10. 멀리 있는 표적을 센서 고장으로 판단한 문제

초기에는 ECHO가 제한시간 안에 돌아오지 않으면 모두 센서 timeout으로 처리해 정상적인 빈 공간에서도 차량이 정지할 수 있었다.

해결:

- 30 ms 안에 ECHO가 없으면 **NO_ECHO**로 분류한다.
- **NO_ECHO**는 최대 거리 4.0 m의 빈 공간으로 취급한다.
- 태스크 정지나 갱신 중단으로 데이터가 stale이면 별도로 STOP한다.
- 0.35 m 미만에서 STOP하고, 0.45 m 이상을 3회 확인한 뒤 해제한다.

현재 배선:

HC-SR04 STM32	
TRIG	PA5 GPIO
ECHO	PB3 / TIM2_CH2 입력 캡처

PB3는 STM32F429ZI 데이터시트상 5 V tolerant 디지털 입력이므로, 보드와 센서가 함께 전원 공급되고 공통 GND가 연결된 현재 조건에서는 ECHO를 직접 연결한다. 분압기는 선택 사항이다.

남은 제약:

- 단일 HC-SR04에서는 정말로 먼 표적과 ECHO 선 단선을 완전히 구분하기 어렵다.
- 현재 구현은 지속적인 ECHO 단선도 **NO_ECHO**로 볼 수 있다.
- 초음파가 안전 핵심 센서가 된다면 별도 센서 health check 또는 이중 센서가 필요하다.

8. 서보와 조향 트러블슈팅

T11. 1500 us가 직진이 아니었던 문제

초기에는 RC 서보의 일반적인 중앙값 1500 us를 사용했지만 실제 OrinCar 서보호와 링크에서는 바퀴가 오른쪽으로 치우쳤다. 이후 1300 us도 시험했고, 최종 실차 직진값은 1215 us로 확정했다.

최종 raw 범위:

```
750 us  = 왼쪽 끝
1215 us = 직진
1680 us = 오른쪽 끝
```

교훈:

- 데이터시트의 일반적인 1000~2000 us 또는 1500 us 중앙값을 최종 차량값으로 간주하면 안 된다.
- 서보축 각도보다 최종 바퀴 방향을 기준으로 중앙과 끝값을 정해야 한다.
- 기계식 스톱퍼에 닿아 소음이나 떨림이 발생하는 PWM은 사용 범위에서 제외한다.

서보 전원:

- MG996R 전원은 12 V 배터리에서 5 V로 강압해 공급한다.
- 서보 전류를 STM32 보드 5 V 핀에서 공급하지 않는다.
- 배터리 음극, 강압모듈 GND, 서보 GND, STM32 GND를 공통으로 연결한다.
- PB4는 PWM 신호만 전달한다.

T12. 진단 종료 후 자동 중앙복귀

증상:

- raw PWM으로 원하는 위치까지 이동
- 시험 시간이 끝나면 바퀴가 중앙으로 힘있게 복귀
- PWM을 끈 것으로 생각했지만 실제로는 1215 us가 다시 출력됨

원인:

```
servo-calibrate 종료
→ 0 us 진단 종료 명령
→ ControlTask가 진단 모드 종료
→ 일반 목표 조향각 0도 적용
→ 1215 us 출력
```

해결:

- 진단 중 0us는 TIM3_CH1 PWM을 실제로 정지한다.
- 진단 종료 후에는 중앙 명령을 자동으로 재출력하지 않는다.
- 이후 실제 주행 모드가 시작되면 PWM을 다시 시작한다.
- 일반 CMD_STOP은 기존처럼 바퀴를 중앙으로 복귀시킨다.

PWM OFF 후 바퀴가 천천히 움직인다면 소프트웨어 중앙 명령이 아니라 서보 토크가 사라져 링크 힘에 밀리는 물리 현상일 수 있다.

T13. 조향센서 없이 최종 각도 LUT 생성

AS5600/PP-A818이 장착되지 않아 실제 바퀴각을 STM32가 읽을 수 없었다. 서보 펄스와 좌우 앞바퀴각을 7개 지점에서 직접 측정했다.

PWM	왼쪽 바퀴각	오른쪽 바퀴각	등가 중심각
750	+15도	+22도	+19.55도
905	+10도	+17도	+14.18도
1060	+8도	+12도	+10.27도
1215	0도	0도	0도
1370	-8도	-6도	-7.09도
1525	-17도	-18도	-17.56도
1680	-29도	-26도	-28.69도

비어 있는 각도는 인접한 두 LUT 점 사이에서 비례식으로 계산한다.

```
ratio = (target_angle - angle_0) / (angle_1 - angle_0)
```

```
pulse = pulse_0 + ratio * (pulse_1 - pulse_0)
```

예:

```
+5도 → 약 1140 us
-5도 → 약 1324 us
+10도 → 약 1064 us
-10도 → 약 1413 us
```

전체 범위를 하나의 **us/degree**로 계산하지 않고 각 구간마다 별도 비례식을 사용한다. 이를 구간별 선형 보간이라고 한다.

상세 계산, 안티 아커만 분석 및 구현 위치:

- [조향센서 없이 MG996R 7점 LUT를 만든 상세 기록](#)

현재 상태는 임시 해결이다. LUT는 실제 피드백이 아니라 명령 기반 추정이므로 충격, 유격, 하중 또는 타이어 미끄러짐을 감지할 수 없다.

9. 차량 치수와 오도메트리 트러블슈팅

T14. 매뉴얼의 250 mm와 148 mm를 사용할 수 없는 이유

OrinCar 매뉴얼의 250 mm와 148 mm는 전체 길이와 전체 폭이다. 아커만 오도메트리에 필요한 값은 다음 두 치수다.

- 뒷차축 중심부터 앞 조향축 중심선까지의 휠베이스
- 왼쪽/오른쪽 앞바퀴 조향축 중심 간 거리

실차에서 직접 측정한 값:

```
wheelbase = 135 mm = 0.135 m
front steering-pivot distance = 85 mm = 0.085 m
```

전체 차폭에는 타이어 폭과 허브 돌출이 포함되므로 조향축 간격 대신 사용할 수 없다.

T15. 형상값을 넣어도 오도메트리가 갱신되지 않는 문제

현재 상태:

- 휠베이스와 조향축 간격은 **vehicle_config.h**에 반영됨
- 엔코더는 823 count/rev로 보정됨
- 7점 조향 명령 LUT가 적용됨
- 물리 조향센서는 없음

그러나 현재 **OdometryTask**는 다음 조건을 요구한다.

```
encoder_calibrated == true
steering_angle_valid == true
```

조향센서가 없으므로 `steering_angle_valid`가 false이고 위치 적분은 계속 비활성화된다. 형상 보정 플래그를 켜는 사실만으로 오도메트리가 활성화되는 것은 아니다.

또한 현재 V2에서 `0x82`는 `COMMAND_RESULT`이므로 구형 V1 `TELEMETRY_ODOMETRY`를 그대로 송신할 수 없다.

후속 작업:

1. 목표 등가 중심각을 명령 기반 오도메트리 입력으로 연결
2. 해당 입력이 실제 센서가 아닌 `estimated`임을 상태에 표시
3. V2의 비충돌 Message ID로 `x`, `y`, `yaw`, `distance`, `curvature` 송신
4. 뒤집힌 차량에서 부호와 증가 방향 검증
5. 바닥 저속 주행에서 직선거리와 회전반경 검증

IMU는 아직 미장착이지만 기본 모터/조향/엔코더/UART 시험의 필수 조건은 아니다. IMU가 추가되면 아커만 yaw rate 와 IMU yaw rate를 비교하고 누적 방향 오차를 보정한다.

10. PC-as-Jetson 통합 시험

PC 시험은 진단용 raw PWM만 확인하는 시험이 아니다. 실제 Jetson과 동일한 `CMD_DRIVE`, 20 Hz heartbeat, 속도 PI, 조향 LUT, 텔레메트리 및 정지 경로를 사용한다.

사전 조건:

- 차량을 뒤집어 구동 바퀴가 모두 공중에 있도록 함
- HC-SR04 앞 45 cm 이상을 비움
- 12 V 모터 전원, 서보 5 V 강압 전원 및 STM32 GND 공통 확인
- 최신 펌웨어 플래시 후 보드 리셋
- PC 벤치에서는 `VEHICLE_COMM_USE_STLINK_VCP=1U`

전체 시험:

```
py tools\uart_protocol_test.py drive-scenario `
--port COM11 `
--wheels-off-ground
```

시나리오:

```
neutral 재무장
→ 전진 직진
→ 전진 좌회전
→ 전진 우회전
→ 후진 직진
→ heartbeat 중단
→ 300 ms 자동정지 확인
```

```
→ neutral 재무장
→ CMD_STOP
```

뒤집힌 상태에서 검증 가능한 것:

- UART 프레임, CRC, SEQ
- 전진/후진 PWM
- 엔코더 Count와 속도 부호
- 조향 명령과 LUT 방향
- watchdog과 재무장
- 안전 상태와 fault bits

바닥에서만 검증 가능한 것:

- 실제 이동거리
- 실제 회전반경
- 직진 편향
- 하중 상태 속도 PI
- 타이어 미끄러짐
- 실제 오도메트리 정확도

11. 현재 남아 있는 미검증 항목

항목	현재 상태	필요한 시험
조향 실제 피드백	센서 없음, LUT 추정	향후 센서 장착 또는 바닥 회전반경
명령 기반 오도메트리	아직 연결 안 됨	코드 연결 후 PC/바닥 시험
V2 오도메트리 텔레메트리	Message ID 미확정	프로토콜 확장
타이어 유효 구름 둘레	64 mm 명목값 사용	하중 상태 1 m 주행
PI 최종 이득	현재 Kp=120, Ki=20	실제 차량 단계응답
UART5 물리 배선	COM11은 통과 가능, UART5 별도	3.3 V USB-UART Echo
Jetson 실제 연동	PC 대체 시험 준비	Jetson TX/RX/GND 연결
IMU	미장착	장착 방향, 바이어스, yaw rate
물리 E-stop	핀/차단 경로 미확정	독립 에너지 차단 설계
초음파 단선 진단	NO_ECHO와 구분 어려움	health check 또는 이중화

12. 재발 방지 체크리스트

새 기능이나 배선을 추가할 때 다음을 순서대로 확인한다.

1. .ioc에서 핀과 Alternate Function을 확인한다.
2. STM32 MCU 핀만 보지 말고 DISC1 보드 회로도의 공유 장치를 확인한다.
3. .ioc, MSP 초기화, HAL Handle, 드라이버 채널 및 실제 배선을 함께 바꾼다.
4. 전원 전압뿐 아니라 모든 디지털 신호의 공통 GND를 확인한다.

5. UART **ACCEPTED**와 물리 동작 성공을 구분한다.
6. PC 도구, wire protocol 및 플래시된 펌웨어 버전을 함께 관리한다.
7. 구조체 payload 길이는 상수 중복보다 **sizeof/calcsize**로 검증한다.
8. 한 번의 엔코더 회전보다 여러 회전과 양방향 평균을 사용한다.
9. 데이터시트의 대표값보다 최종 조립 차량의 실측값을 우선한다.
10. 뒤집힌 차량 시험과 바닥 주행 시험이 검증하는 범위를 구분한다.
11. 센서 추정값은 실제 측정값과 동일한 유효 플래그로 보고하지 않는다.
12. 해결된 문제와 아직 미검증인 항목을 같은 완료 상태로 기록하지 않는다.

13. 근거 문서와 코드

- [Jetson-STM32 UART V3 인터페이스](#)
- [초음파 안전 설계](#)
- [조향과 오도메트리 작업 기록](#)
- [조향센서 없는 7점 LUT 상세 기록](#)
- [2026-07-28 모터/엔코더 인수인계](#)
- [App/Inc/vehicle_config.h](#)
- [App/Src/task_control.c](#)
- [App/Src/task_odometry.c](#)
- [Core/Src/comm_service.c](#)
- [Drivers/BSP/Src/motor.c](#)
- [Drivers/BSP/Src/encoder.c](#)
- [Drivers/BSP/Src/steering_sensor.c](#)
- [tools/uart_protocol_test.py](#)

관련 Git 이력:

```
e8acfdc  BTS7960 LPWM을 PA3/TIM5_CH4로 변경
d757698  PI 및 초음파 NO_ECHO 상태 처리 개선
4fff0cd  조향 센싱과 아커만 오도메트리 구조 추가
5fc1a80  MG996R 진단과 실차 펄스 보정 추가
61fcc61  PC-as-Jetson 통합 시험과 조향 명령 추정 허용
```